Insertion Sort

Ordenamiento por inserción: explicación detallada del algoritmo y de su visualización con bolos en Graph Maker

1. ¿Qué es y qué hace?

Insertion Sort (ordenamiento por inserción) construye el resultado ordenado **de a un elemento a la vez**. Mantiene al inicio del arreglo una zona ya ordenada, toma el siguiente elemento de la zona sin ordenar y lo **inserta en el lugar correcto** dentro de la zona ordenada, desplazándolo hacia la izquierda mientras sea menor que su vecino.

Analogía

Es la forma en la que mucha gente ordena las cartas de una baraja en la mano: tomas una carta nueva y la vas deslizando hacia la izquierda hasta que queda entre una carta menor y una mayor. Las cartas que ya tenías en la mano siguen ordenadas.

2. Funcionamiento paso a paso

- Se considera que el primer elemento (posición 0) es, por sí solo, una zona ordenada de tamaño 1.
- Para i desde 1 hasta n - 1, el elemento de la posición i es el que se va a insertar.
- Se compara con el elemento de su izquierda. Si el de la izquierda es **mayor**, se intercambian y el elemento "retrocede" una posición.
- Se repite la comparación con el nuevo vecino de la izquierda. Se detiene cuando el vecino es menor o igual, o cuando llega a la posición 0.
- Ahora la zona ordenada tiene un elemento más. Se pasa al siguiente i.

3. Ejemplo resuelto

Se ordena el arreglo **[5, 3, 8, 1, 4]**. Al comenzar, la zona ordenada es [5].

Vuelta	Qué ocurre	Arreglo resultante
Inicio	Zona ordenada: [5].	[5, 3, 8, 1, 4]
i = 1	Se inserta el 3. Como 5 > 3, se intercambian. Llega a la posición 0.	[**3, 5**, 8, 1, 4]
i = 2	Se inserta el 8. Como 5 > 8 es falso, no se mueve.	[**3, 5, 8**, 1, 4]
i = 3	Se inserta el 1. Se intercambia con 8, luego con 5 y luego con 3. Llega a la posición 0 (3 intercambios).	[**1, 3, 5, 8**, 4]

Vuelta	Qué ocurre	Arreglo resultante
i = 4	Se inserta el 4. Se intercambia con 8 y con 5. Al compararse con 3, 3 > 4 es falso y se detiene (2 intercambios).	[**1, 3, 4, 5, 8**]

Observa: cuando el elemento nuevo ya es mayor que su vecino (como el 8 en i = 2), el algoritmo termina esa vuelta con una sola comparación. Por eso Insertion Sort es muy rápido con datos casi ordenados.

Cómo se visualiza en la aplicación

En la pantalla cada número del arreglo se dibuja como un **bolo de boliche**. El tamaño del bolo es proporcional a su valor: el bolo más pequeño representa el valor mínimo y el más grande el valor máximo. El número aparece dentro del bolo. Se pueden usar entre 1 y 20 bolos, con valores entre 1 y 99.

Los controles de la pantalla son los mismos en los cuatro algoritmos:

- **Slider (barra superior):** elige la cantidad de bolos (1 a 20). Genera la secuencia 1, 2, 3, ... n. Se deshabilita mientras se ordena.
- **Cronómetro:** muestra el tiempo en formato mm:ss.d. Se actualiza cada 100 ms con un Timer y se detiene cuando termina el ordenamiento.
- **Mezclar:** desordena los valores actuales con shuffle().
- **Aleatorio:** genera valores aleatorios entre 1 y 99 (pueden repetirse).
- **Ingresar:** permite escribir los valores a mano, separados por coma o espacio, con validación (solo números, de 1 a 99, máximo 20 valores).
- **Guardar / Cargar:** guarda y recupera listas de valores. El grupo de guardado es compartido, así que lo que se guarda en un algoritmo se puede cargar en los otros.
- **Resolver:** inicia el algoritmo con animación.
- **Velocidad:** el desplegable multiplica las pausas de la animación (y la duración con la que los bolos suben y bajan). Se puede cambiar incluso mientras se ordena.

Variables de estado que usa la animación

- **_actual:** el bolo que se está insertando. Se actualiza a medida que retrocede.
- **_comparado:** el vecino de la izquierda con el que se compara.
- **_fijos:** tamaño de la zona ordenada al inicio del arreglo.
- **_ordenando:** indica si el algoritmo está corriendo.

Colores en la pantalla Insertion Sort

Color del bolo	Significado en la pantalla
Naranja	El bolo que se está insertando (_actual).
Amarillo	El vecino con el que se compara (_comparado).

Color del bolo	Significado en la pantalla
Verde claro	La zona ya ordenada entre sí al inicio del arreglo.
Blanco	Bolos que todavía no se han insertado.

> **Importante:** aquí el verde significa "ordenado entre sí", no necesariamente "en su posición final". Un bolo de la zona verde todavía puede moverse a la derecha cuando llegue un elemento menor que él.

Los bolos _actual y _comparado se elevan ligeramente. Hay una pausa de 120 ms al comparar y de 350 ms al intercambiar (multiplicadas por el factor de velocidad).

4. Código clave

```
for (int i = 1; i < n; i++) {
  int k = i;                    // posición del bolo que se inserta

  while (k > 0) {
    if (_valores[k - 1] > _valores[k]) {
      final temp = _valores[k - 1];     // intercambio con el vecino
      _valores[k - 1] = _valores[k];
      _valores[k] = temp;
      k--;                              // el bolo retrocede
    } else {
      break;      // ya está en su lugar dentro de la zona ordenada
    }
  }
}
```

5. Complejidad

Característica	Valor	Explicación
Mejor caso	O(n)	Arreglo ya ordenado: cada elemento solo se compara una vez con su vecino.
Caso promedio	$O(n^2)$	Cada elemento retrocede, en promedio, la mitad de la zona ordenada.
Peor caso	$O(n^2)$	Arreglo en orden inverso: cada elemento debe retroceder hasta el inicio.
Memoria extra	O(1)	Ordena en el mismo arreglo (in-place).
¿Estable?	Sí	Solo intercambia si el vecino es estrictamente mayor, por lo que los elementos iguales mantienen su orden relativo.

6. Ventajas y desventajas

Ventajas

- Muy simple de implementar.
- Excelente con listas pequeñas o casi ordenadas (llega a O(n)).
- Es estable y no necesita memoria extra.
- Es "adaptativo": su velocidad mejora cuanto más ordenados estén los datos.
- Funciona bien cuando los datos llegan de a poco, porque puede insertar sobre la marcha.

Desventajas

- Con muchos datos desordenados es lento: $O(n^2)$.
- Hace muchos intercambios: un elemento muy pequeño al final debe recorrer todo el arreglo, una posición a la vez.

7. ¿Cuándo usarlo?

Es una buena opción para pocos datos o para datos que ya están casi ordenados. Muchas librerías lo usan internamente para ordenar fragmentos pequeños dentro de algoritmos más rápidos. También es la base de Shell Sort, que lo mejora permitiendo mover los elementos en saltos más grandes.